

Polymorphism also refers to *ad hoc polymorphism* or *operator overloading*, where operators (such as `+` or `*`) can have different behavior based on the type of objects they're operating on. For example, the `+` operator does mathematical addition when operating on two integer or float values, but it does string concatenation when operating on two strings. Operator overloading is covered in Chapter 17.

When Not to Use Inheritance

It's easy to overengineer your classes using inheritance. As Luciano Ramalho states, "Placing objects in a neat hierarchy appeals to our sense of order; programmers do it just for fun." We'll create classes, subclasses, and sub-subclasses when a single class, or a couple of functions in a module, would achieve the same effect. But recall the Zen of Python tenet in Chapter 6 that *simple is better than complex*.

Using OOP allows you to organize your code into smaller units (in this case, classes) that are easier to reason about than one large `.py` file with hundreds of functions defined in no particular order. Inheritance is useful if you have several functions that all operate on the same dictionary or list data structure. In that case, it's beneficial to organize them into a class.

But here are some examples of when you don't need to create a class or use inheritance:

- If your class consists of methods that never use the `self` or `cls` parameter, delete the class and use functions in place of the methods.
- If you've created a parent with only a single child class but never create objects of the parent class, you can combine them into a single class.
- If you create more than three or four levels of subclasses, you're probably using inheritance unnecessarily. Combine those subclasses into fewer classes.

As the non-OOP and OOP versions of the tic-tac-toe program in the previous chapter illustrate, it's certainly possible to not use classes and still have a working, bug-free program. Don't feel that you have to design your program as some complex web of classes. A simple solution that works is